

Never FADE Away: Distributional Emulation

Jack Fraser-Govil, Sam Watson, Penny Hancock

August 4, 2026

Contents

1	Model Statement	2
1.1	Motivation	2
1.2	The Data	2
1.3	Generating Data	3
1.4	Defining the FADE	4
2	Mathematical Niceties	7
2.1	Mean Subtraction	7
2.2	Parameter Interpolation vs Distribution Interpolation	7
2.3	Generalised Expectation Maximisation	9
2.4	Priors	13
3	The FADE User Manual	13
3.1	Installing the Code	13
3.2	Compiling the Code	13
3.3	FADE and JSL	14
3.4	Training Models	15
3.5	Using Models	16

1 Model Statement

1.1 Motivation

There are many decades of literature regarding the problem of estimating a mapping $\mathcal{P} \rightarrow \mathcal{O}$, from some parameter space $\mathcal{P}$ to an observable space $\mathcal{O}$, conditioned on a set of observed data $\mathbf{T} = \{(\mathbf{y} \in \mathcal{O}, \mathbf{x} \in \mathcal{P})\}$. The vast majority of this work is on ‘point predictors’, wherein one is able to reliably compute $\mathbb{E}(\hat{\mathbf{y}}|\mathbf{w}, \mathbf{T})$ for some new proposed set of covariates $\mathbf{w} \in \mathcal{P} \setminus \mathbf{T}$.

More modern methods, such as Gaussian Process Emulation, are able to also predict a distribution, but in doing so must assume a fixed parametric family which is attached to the point predictor.

For our purposes however, we are more interested in a *distributional predictor*, which would enable us to predict not just the behaviour of the mean, but the entire distribution, $p(\hat{\mathbf{y}}|\hat{\mathbf{x}}, \mathbf{T})$.

Whilst there are many methods which allow us to *quantify the uncertainty* of a prediction, there are few which allow us to fully reconstruct an accurate *distribution*, if the underlying ‘truth’ that is being distributed is genuinely a stochastic process, rather than a ‘deterministic process with unknown noise’.

1.2 The Data

We suppose that there exists some observable $\mathbf{y} \in \mathbb{R}^q$ which may be extracted from a function $\hat{M}$ evaluated at a parameter vector $\mathbf{P} \in \Pi$:

$$\mathbf{y} = \hat{M}(\mathbf{P}) \quad (1)$$

For the cases that interest us, $\hat{M}$ is a simulational (though potentially stochastic) model of a system where the input dimension, $\dim(\Pi)$ is large.

1.2.1 Parameter Promotion

For the purposes of a statistical emulation, we suppose that this full parameter space may be divided into two subspaces:

1. **Conditional Variables** These are the covariates of interest for the final emulator. They define the *emulation space*

$$\mathcal{E} = \mathbb{R}^n \quad (2)$$

and are denoted by $\mathbf{x}$.

2. **Latent Variables** The remaining parameters of the mechanistic model. These variables are not explicitly represented in the emulator and instead contribute stochastic variability to the output distribution. They are denoted $\mathbf{z}$, and we assume that there exists some prior on their values, which may be conditioned on the covariates $P(\mathbf{z}|\mathbf{x})$.

Why use manual promotion?

PCA and related nonlinear projection methods provide a method in which the machine computes a projection into the space of the ‘most important variables’:

$$\hat{\chi} = \mathcal{T}(\mathbf{P}) \quad (3)$$

It would then be possible to learn $p(\mathbf{y}|\hat{\chi})$, which is an easier task since $\hat{\chi}$ is a lower-dimensional entity, and the projection has (in theory) captured the bulk of the underlying correlations.

However, this approach requires specifying a full parameter vector $\mathbf{P}$ whenever the model is used to generate a prediction. For complex mechanistic systems this is often undesirable. For example, in the mosquito model we wish to predict outcomes conditioned on the gene drive parameters, without needing to specify rainfall realisations and mosquito lifetimes.

In short, we wish to simultaneously marginalise over these parameters, and fit the covariates of interest.

1.3 Generating Data

1.3.1 Latent Variables & Priors

The ‘obvious’ solution for generating the data we need for our emulator would be to sample $\mathbf{x}$ uniformly from $\mathcal{E}$, and $\mathbf{z}$ from the latent prior, $P(\mathbf{z})$, and then compute $\mathbf{y} = \hat{M}(\mathbf{x}, \mathbf{z})$. Our dataset would then be $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^D$.

However, this would have the major drawback that if – at a later date – our priors on the latent variables were updated, then we would need to completely regenerate the training data. This would be a significant bottleneck in computation time: for the models of interest to us, the vast majority of the computational expense will be in creating $\mathcal{D}$.

Instead, we suppose that we maintain two datasets, where the latent priors are drawn from $P_{\text{sample}}(\mathbf{z}, \mathbf{x})$, which is not necessarily the same as our current prior on $\mathbf{z}$.

$$\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i, \eta_i)\}_{i=1}^D \quad \mathcal{Z} = \{\mathbf{z}_i, \zeta_i\}_{i=1}^D \quad (4)$$

The ordering between these sets is the same (so $\mathbf{y}_i = \hat{M}(\mathbf{x}_i, \mathbf{z}_i)$ is true). The parameters η_i and ζ_i are defined as:

$$\begin{aligned} \eta_i &= P_{\text{sample}}(\mathbf{z}_i) \\ \zeta_i &= \frac{P_{\text{latent}}(\mathbf{z}_i)}{\eta_i} \end{aligned} \quad (5)$$

Where P_{latent} is the *current prior on the latent variables*, and we recall P_{sample} is the distribution used to generate $\{\mathbf{z}\}$. This means that $\mathcal{D}$ is immutable (unless more data is added), and changes to the prior require simply recomputing ζ_i - a trivial task.

When computing the log-posterior conditioned on the observed evidence, we simply need to correct by a factor ζ_i :

$$\mathcal{L}(\mathcal{D}, \mathcal{Z}) = \sum_{i=1}^D \zeta_i \times \ln p(\mathbf{y}_i | \mathbf{x}_i, \dots) \quad (6)$$

We can see that if $\zeta_i = 1$ (which occurs when $P_{\text{sample}} = P_{\text{latent}}$), then this reduces to the obvious case. Otherwise, it means that we have an effective sample size

$$D^* = \frac{(\sum_i \zeta_i)^2}{\sum_i \zeta_i^2} \quad (7)$$

The choice of sampling distribution must therefore be informed on a best-guess understanding of how the latent prior may evolve to ensure that D^* is a reasonable size for both the current latent prior, and any future updates. If $D^* \ll D$, then it will be necessary to generate more data, which is to be avoided under the current paradigm.

1.3.2 x-Clustering

We also note that clustering the data at values of identical $\mathbf{x}$ provides both a vast computational speed up, and negates a danger posed by the mean-fitting discussed in §2.1. That is, instead of generating D unique values of $\mathbf{x}$ (and then $\mathbf{z}$ and hence $\mathbf{y}$), we suppose that only T values of $\mathbf{x}$ are unique.

At each unique $\mathbf{x}_t$, we then generate A_t pairs of $(\mathbf{z}, \mathbf{y})$ pairs¹, to form a clustered dataset

$$\mathcal{D} = \left\{ \mathbf{x}_t, \{\mathbf{y}_t^a, \eta_t^a\}_{a=1}^{A_t} \right\}_{t=1}^T \quad (8)$$

This provides a speed up since the bulk of the computation time during training is spent computing $W_i(\mathbf{x})$ - if this value can be reused $\approx A_t$ times, it provides a commensurate $\approx A_t \times$ -speedup.

1.4 Defining the FADE

1.4.1 Model Ingredients

We now review several model classes that provide useful components for constructing a probabilistic emulator.

Partition Models A partition model divides the covariate space into a collection of disjoint regions, each associated with a unique statistical model. A common construction uses a Voronoi Tessellation and fits a local correlation ellipsoid. The main downside of these models is that they are inherently discontinuous: the distribution is identical for any two points which share a cell.

Mixture of Experts Alternatively, a mixture of experts may be used, in which either the conditional distribution is represented as a weighted sum of distributions associated with an ‘expert’:

$$p(\mathbf{y}|\mathbf{x}) = \sum_i^{N_e} w_i(\mathbf{x}) p_i(\mathbf{y}) \quad (9)$$

Alternatively, if we assume that each p_i is parameterised by some vector $\boldsymbol{\xi}_i$, such that $p_i(\mathbf{y}) = f(\mathbf{y}; \boldsymbol{\xi}_i)$ we may instead weight this to give a consensus distribution:

$$p(\mathbf{y}|\mathbf{x}) = f\left(\mathbf{y}; \sum_i w_i(\mathbf{x}) \boldsymbol{\xi}_i\right) \quad (10)$$

The weights which determine ‘how much’ each expert contributes to the mixture at each position in covariate space is typically derived from a Neural Network with a gating layer at the output.

This means they come with many of the ‘black box’ issues that have plagued machine learning methods since their introduction: it is difficult to understand why one region of covariate space may be assigned to one expert.

Kernel Interpolators A simpler and more geometrically interpretable weighting scheme can be constructed using kernels. Suppose each expert is associated with a location $\mathbf{c}_i$ in the covariate space. We define weights using a radial basis function kernel:

$$w_i(\mathbf{x}) = \frac{K(d(\mathbf{x}, \mathbf{c}_i | G))}{\sum_\ell K(d(\mathbf{x}, \mathbf{c}_\ell | G))}. \quad (11)$$

¹Indeed, if the model is stochastic, it may be that we keep $\mathbf{z}$ constant and sample several $\mathbf{y}$ from a single $(\mathbf{x}_t, \mathbf{z}_a)$ pair - the optimal distribution here is to be determined by the trainer.

Here

$$d(\mathbf{x}, \mathbf{y} \mid G) = \sqrt{(\mathbf{x} - \mathbf{y})^\top G (\mathbf{x} - \mathbf{y})} \quad (12)$$

is a Mahalanobis distance induced by the metric G , and $K(d)$ is a radial kernel satisfying $K(0) = 1$ and decaying with increasing distance. Under this construction the prediction at $\mathbf{x}$ is obtained by smoothly interpolating between nearby experts. If $\mathbf{x}$ lies close to $\mathbf{c}_k$ then $w_k(\mathbf{x}) \approx 1$ and the prediction is dominated by the k^{th} expert.

1.4.2 Department-of-Expert Models

A Department-of-Expert Model combines the geometric structure of partition models with the smooth interpolation properties of kernel mixtures. Such a model is defined by:

1. **N_d departments:** A set of partitions which define the local covariate topology of a region
2. **N_e experts:** A set of distribution functions assigned to a position in covariate space
3. An interpolation kernel K_E which determined which experts are to be consulted at a given region of space.

A Naive DoE Model In the naive case, we allow a department to have a sharp boundary, such that no information can transfer across its boundary. In the case where $N_e = N_d$ and there is one expert per cell, this is precisely the Partition Model introduced above. In cases where $N_e > N_d$, this is a per-Cell Kernel Interpolator.

Making Departments Flexible To avoid the discontinuities associated with the hard partitions, we assume that rather than each point $\mathbf{x}$ belonging to a single cell, there is a weighted membership of the k^{th} cell, $D_k(\mathbf{x})$, according to:

$$D_k(\mathbf{x}) = \frac{K_D(d_k(\mathbf{x}, \mathbf{s}_k))}{\sum_j K_D(d_j(\mathbf{x}, \mathbf{s}_j))} \quad (13)$$

Here K_D is the ‘department kernel’ a distinct radial basis function that we choose *a priori*.

Making Covariate Space Anisotropic The distance too is a function of the cell it is being measured in², via the unique metric associated with each cell:

$$d_k(\mathbf{x}, \mathbf{y}) = \sqrt{(\mathbf{x} - \mathbf{y})^\top G_k (\mathbf{x} - \mathbf{y})}. \quad (14)$$

If G_k is restricted to a diagonal form (as in [Payne et al.](#)) this simply rescales the coordinate axes, assigning different relative importance to each covariate within the k^{th} region. Relaxing this restriction allows G_k to contain off-diagonal terms, which encode correlations between covariates and therefore rotate and stretch the local coordinate system. Consequently each department does not merely partition the parameter space but instead defines a local *correlation topology*.

²Recalling that each cell is, strictly speaking, now infinite in extent

$$p^{\text{fade}}(\mathbf{y}|\mathbf{x}, \mathbf{\Delta}, \mathbf{\Theta}, \boldsymbol{\eta}) = \mathcal{F} \left[y \left| \sum_{i=1}^{N_e} W_i(\mathbf{x}) \boldsymbol{\xi}_i \right. \right] \quad (15)$$

$$W_i(\mathbf{x}) = \sum_{k=1}^{N_d} D_k(\mathbf{x}) w_i^k(\mathbf{x}) \quad (16)$$

$$w_i^k(\mathbf{x}) = \frac{D_k(\mathbf{c}_i) K_e(d_k(\mathbf{x}, \mathbf{c}_i))}{\sum_{j=1}^{N_e} T_k(\mathbf{c}_j) K_e(d_k(\mathbf{x}, \mathbf{c}_j))}, \quad (17)$$

$$D_k(\mathbf{x}) = \begin{cases} 1 & N_d = 1 \\ \frac{K_d(d_k(\mathbf{x}, \mathbf{s}_k))}{\sum_{j=1}^{N_d} K_d(d_j(\mathbf{x}, \mathbf{s}_j))} & \text{else} \end{cases}, \quad (18)$$

$$d_k(\mathbf{x}, \mathbf{y}) = \sqrt{(\mathbf{x} - \mathbf{y})^\top G_k (\mathbf{x} - \mathbf{y})} \quad (19)$$

Where:

1. $\mathbf{y} \in \mathcal{Y} \subset \mathbb{R}^q$ is the value to be emulated. The space $\mathcal{Y}$ is constrained to be a subset of $\mathbb{R}^q$ by the choice of $\mathcal{F}$ and the prior.
2. $\mathbf{x} \in \mathcal{E} \subset \mathbb{R}^n$ is the covariate input vector
3. $\mathbf{\Delta} = (N_d, N_e)$ are our discrete model parameters
 - (a) N_d is the number of Voronoi seeds used (due to the anisotropic metric, the space may be split into $n \geq N_d$ disjoint regions)
 - (b) N_e is the number of experts populating the space
4. $\boldsymbol{\eta} = (K_e, K_d, \mathcal{F})$ are the model hyperparameters - our final result will be conditioned on these values.
 - (a) $K_d(d)$ is the *expert kernel*, an isotropic distance kernel of some kind, used to determine how quickly the information is exchanged between experts in the same department
 - (b) $K_e(d)$ is the *department kernel*, an isotropic distance kernel used to determine how easily experts can communicate between departments
 - (c) $\mathcal{F}$ is a family of probability distribution functions that each expert may choose from.
5. $\mathbf{\Theta} = \left(\left\{ \mathbf{s}_k, \{\phi_{ij}^k\} \right\}_{k=1}^{N_p}, \left\{ \mathbf{c}_i, \boldsymbol{\xi}_i \right\}_{i=1}^{N_e} \right)$ specifies our partitioned-expert scheme
 - (a) $\mathbf{s}_k \in \mathcal{E}$ are the coordinates seeds of the k^{th} Voronoi pseudo-partition
 - (b) $\{\phi_{ij}^k\}_{0 \leq j \leq i \leq n}^{0 \leq j \leq i}$ are a set of parameters ($n(n+1)/2$ of them) which are used to define a lower triangular matrix with positive diagonal elements:

$$[L_k]_{ij} = \begin{cases} \exp(\phi_{ij}^k) & i = j \\ \phi_{ij}^k & i > j \\ 0 & \text{else} \end{cases} \quad (20)$$

This then defines a unique metric, $G_k(\{\phi^k\}) = L_k L_k^\top$, which is guaranteed to be positive definite and symmetric. ϕ_{ij}^k is otherwise unconstrained

- (c) $\mathbf{c}_i \in \mathbb{R}^n$ are the coordinates of the i^{th} expert.
- (d) $\boldsymbol{\xi}_i$ is the parameter set which specifies p_i in the family $\mathcal{F}$:

$$p_i(\mathbf{y}) = \mathcal{F}(\mathbf{y}; \boldsymbol{\xi}_i) \quad (21)$$

Table 1: The full specification of a FADE model.

2 Mathematical Niceties

The model specificaiton in Table 1 explains how, given a set of parameters $\alpha = (\Delta, \eta, \Theta)$, we may generate a probability distribtion function, $p(\mathbf{y}|\alpha)$. This section details the mathematical

2.1 Mean Subtraction

The FADE predictor as it currently stands delegates two tasks to the experts: a pointwise interpolation of the bulk behaviour of the function $\hat{M}$ to be emulated, and the subsequent distribution around that value.

This means that the number of experts required to capture the behaviour scales with *both* the complexity of the function's mean, and with the complexity of its error structure - though in the case of the first, it translates as multiple copies of the same distribution, shifted by a constant, in order to capture the varying mean.

We can reduce the number of experts required to capture this behaviour by estimating the mean behaviour of the input data, subtracting it from the training data:

$$\begin{aligned} \mu(\mathbf{x}) &= \mu(\mathbf{x}|\mathcal{D}') \\ \mathcal{D} = \{\mathbf{x}_t, \{\mathbf{y}_t^a, \zeta_t^a\}\} &\rightarrow \mathcal{R} = \{\mathbf{x}_t, \{\mathbf{y}_t^a - \mu(\mathbf{x}_t), \zeta_t^a\}\} \end{aligned} \quad (22)$$

We then train on this residual set, and at prediction time, we shift back by the mean:

$$p^{\text{fade}}(\mathbf{y}, \mathbf{x}, \alpha) = \mathcal{F} \left[\mathbf{y} - \mu(\mathbf{x}) \left| \sum_i^{N_e} W_i(\mathbf{x}) \xi_i \right. \right] \quad (23)$$

We note that we generated the mean function $\mu(\mathbf{x})$ on a different set of data, $\mathcal{D}'$ than that which we then residual-corrected (and then trained the emulator on), $\mathcal{D}/\mathcal{R}$. This ensures that the biases of the mean-predictor are not correlated with the residuals. The ‘mean dataset’ should therefore be drawn from a random subset of the full training data.

We note that since we do *not* subsequently require the FADE to respect $\mu(\mathbf{x})$ as the mean (i.e. we do not require that $\langle \mathcal{F} \rangle = 0$), the emulator is perfectly able to disagree with the mean-correction if it desires, and allocate experts to reconstruct the best-fitting distribution.

That is, if we ‘corrected’ by any arbitrary function $f(\mathbf{x})$, the FADE would still be able to recover the optimal fitting, given enough experts. By correcting by a function which is (a close estimate of) the mean, we aim to minimise the number of experts required.

2.2 Parameter Interpolation vs Distribution Interpolation

When interpolating between an expert-PDF defined by some parameter set ξ_i and assigned a weight W_i , we have a choice of what to interpolate to reach a consensus-PDF, p^{cns} . We could interpolate over the PDFs themselves:

$$p^{\text{cns}}(y) \stackrel{?}{=} \sum_i W_i p(y|\xi_i) \quad (24)$$

Or we may interpolate over the parameter vectors:

$$p^{\text{cns}}(y) \stackrel{?}{=} p \left(y \left| \sum_i W_i \xi_i \right. \right) \quad (25)$$

Each of these has their own advantages and disadvantages - and in certain situations, they are equivalent.

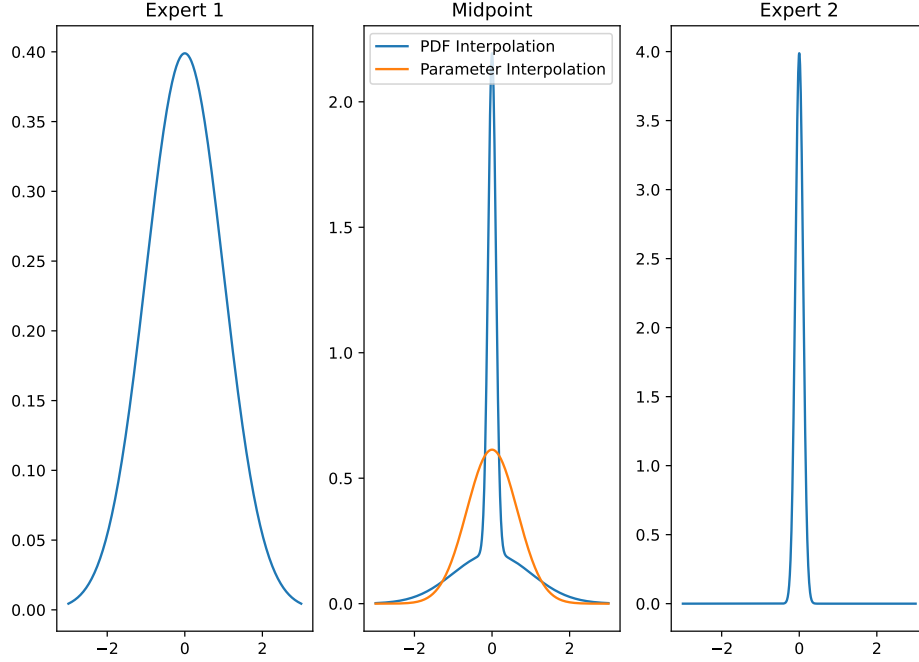


Figure 1: An example of the two different interpolation schemes for Gaussians of differing variance

2.2.1 PDF Interpolation

PDF-Interpolation (24) has the property that the resultant modality is an emergent property of the individual expert PDFs and their proximity to each other; thereby satisfying our requirement that we are not merely assuming the family from which the overall distribution is formed.

However, it has unintuitive interpolation behaviour. Consider the simple 2-expert case, $p_1 = \mathcal{N}(\mu, \sigma_1^2)$ and $p_2 = \mathcal{N}(\mu, \sigma_2^2)$, demonstrated in figure 1. The PDF at the midpoint between these two is $p^{\text{cns}} = 0.5\mathcal{N}(\mu, \sigma_1^2) + 0.5\mathcal{N}(\mu, \sigma_2^2)$: the two peaks ‘ghost’ into each other, rather than gradually morphing into one another: if the behaviour to be fitted was genuinely a gradually shifting variance, we would require a vast number of experts inserted along this midpoint line.

2.2.2 Parameter Interpolation

Parameter-Interpolation (25) solves this unintuitive behaviour; the properties of the parameter distribution gradually morph into one another, by construction.

The downside of it is that we have chosen, *a priori* a model $p(y|\xi)$, and our output will always be of this form. If our family is that of P -modal Gaussian mixture models, the output will always be a P -modal Gaussian mixture: there is no scope for the ‘emergent’ modality seen in the PDF interpolation schema outside of the standard ‘mode deletion’ of weighting a mode to zero. This contrasts with our desire for our distributional emulator to be distinct from a ‘mean predictor with an *a priori* PDF painted on top’ that we expressed in the motivation for this project.

2.2.3 Interpolation Equality

These two methods are equal when the model is a piecewise linear (or piecewise constant) family-free³ distribution, for example:

$$\begin{aligned}
p(y|\mathbf{p}) &= \ell(y; \mathbf{p}) \\
&= \begin{cases} 0 & y \leq y_{\min} \text{ or } y \geq y_{\max} \\ \left(p_i + \frac{(p_{i+1} - p_i)}{\Delta y} (y - y_{\min} - i\Delta) \right) & \text{else} \end{cases} \\
i(y) &= \left\lfloor \frac{y - y_{\min}}{\Delta} \right\rfloor \\
p_0 &= p_{R+1} = 0 \\
\Delta &= \frac{y_{\max} - y_{\min}}{R + 1}
\end{aligned} \tag{26}$$

This is a standard non parametric specification of a curve which is anchored to zero at the endpoints; if the parameters $\mathbf{p} = (p_1, \dots, p_R)$ are positive and normalised such that:

$$\Delta \sum_{i=1}^R p_i = 1 \tag{27}$$

Then this is a valid PDF. It also follows from the linearity, that:

$$\sum_i W_i p(y|\mathbf{p}_i) = p \left(y \left| \sum_i W_i \mathbf{p}_i \right. \right) \tag{28}$$

And thus that PDFs of this form do not matter which interpolation regime is used.

2.2.4 The Mixed Model

This therefore inspires us to generate a model which has the advantages of both forms, whilst (hopefully) minimising the number of parameters. We propose that each expert in the model use a mixture model of a low-resolution family-free PDF, and a P -order Gaussian mixture model:

$$p(y|\xi_i) = \sum_{j=0}^{P-1} \pi_i^j \mathcal{N}(y; \mu_i^j, \sigma_i^{j2}) + \pi_i^P \ell(y; \mathbf{p}_i) \tag{29}$$

Where the consensus is arrived at through Parameter interpolation, with $\Xi = (\{\xi_i\})$

$$\begin{aligned}
p^{\text{cns}}(y|\mathbf{x}, \Xi) &= \sum_j^{P-1} \bar{\pi}_j \mathcal{N}(y; \bar{\mu}_j, \bar{\sigma}_j^2) + \bar{\pi}_P \ell(y; \bar{\mathbf{p}}) \\
\bar{\xi}(\mathbf{x}) &= \sum_i W_i(\mathbf{x}) \xi_i
\end{aligned} \tag{30}$$

2.3 Generalised Expectation Maximisation

We are therefore left with a parameter vector, Ξ which has $U = 3P + 1 + R$ dimensions (a (π, μ, σ^2) for each of the P Gaussian modes, a π for the non parametric mode, which is defined by R elements). Of these

³Or ‘non-parametric’, but there are parameters!

$\tilde{U} = U - 2$ are independent (one degree of freedom being absorbed by the constraint $\sum_i \pi_i = 1$, and another by the constrain that ℓ be normalised to 1).

Our goal is to maximise the posterior probability:

$$L(\Xi; \mathcal{D}) = \left[\prod_{t=0}^T \prod_{a=0}^{A_t} (p^{\text{cns}}(y = y_t^a | \mathbf{x}_t, \Xi))^{\zeta_t^a} \right] p(\Xi) \quad (31)$$

Where we recall that we have grouped our data $\mathcal{D}$ into N_t observations of y for each value of the emulation parameters $\mathbf{x}_t$, weighted by the prior-ratio ζ_t^a (Eq. 5).

It would be possible to take derivatives with respect do Ξ , but mixture models are infamously difficult to converge properly. Instead, we rely on the Generalised Expectation-Maximisation algorithm, which exhibits better convergence behaviour. This only allows us to optimise the *parameters* of each expert, not their positions – so we assume that this is handled by a separate optimisation routine.

We assume that the experts and department positions are fixed per-iteration, and thus that $W_i(\mathbf{x})$ can be computed freely: we define $W_i^t = W_i(\mathbf{x}_t)$ on the training data. We may then define the ‘complete’ logarithm function associated with the latent variable $\mathbf{w} = (\{w_t^a\})$, which has dimensionality D , the total number of datapoints in the training set:

$$L(\Xi, \mathbf{w}) = \prod_{a,t} \prod_{p=0}^P [\bar{\pi}_{p,t} f_p(y_t^a; \bar{\theta}_{p,t})]^{\zeta_t^a \mathbb{I}(w_t^a = p)} \quad (32)$$

Where we have defined:

$$\begin{aligned} f_p(x, \theta_p) &= \begin{cases} \mathcal{N}(x; [\theta]_{2p}, [\theta]_{2p+1}) & 0 \leq p < P \\ \ell(x; \theta) & p = P \end{cases} \\ \dim(\theta_p) &= \begin{cases} 2 & 0 \leq p < P \\ R & p = P \end{cases} \\ \mathbb{I}(x = y) &= \begin{cases} 1 & x = y \\ 0 & \text{else} \end{cases} \end{aligned} \quad (33)$$

This allows use to take the logarithm and make the mixture model separable:

$$\mathcal{L}(\Xi, \mathbf{w}) = \sum_{a,t,p} \mathbb{I}(w_t^a = p) \zeta_t^a [\log(\bar{\pi}_{p,t}) + \log(f_p(y_t^a; \bar{\theta}_{p,t}))] \quad (34)$$

The conditional density of $\mathbf{w}$ - which is also known as the *contribution* that each mode is making towards a give datapoint is:

$$\begin{aligned} C_{p,t,a} &= \zeta_t^a P(w_t^a = p | y_t^a, \Xi) \\ &= \zeta_t^a \frac{\bar{\pi}_{p,t} f_p(y_t^a; \bar{\theta}_{p,t})}{\sum_q \bar{\pi}_{q,t} f_q(y_t^a; \bar{\theta}_{q,t})} \end{aligned} \quad (35)$$

We may then define the Q function - the expectation of the logarithm - as:

$$\begin{aligned} Q(\Xi', \Xi) &= \int d\mathbf{w} \mathcal{L}(\Xi', \mathbf{w}) P(\mathbf{w} | \mathbf{y}, \Xi) \\ &= \sum_{p,t,a} C_{p,t,a}(\Xi) \left(\log(\bar{\pi}_{p,t}') + \log(f_p(y_t^a; \bar{\theta}_{p,t}')) \right) \end{aligned} \quad (36)$$

The (G)EM algorithm relies on the statement that; given an initial set of parameters Ξ , a new set of parameters Ξ' which improves Q also improves $\mathcal{L}$:

$$Q(\Xi', \Xi) \geq Q(\Xi, \Xi) \implies \mathcal{L}(\Xi') \geq \mathcal{L}(\Xi) \quad (37)$$

Given an initial guess Ξ_0 (and hence a set $\{C_{p,t,a}\}$) it suffices therefore to find a Ξ' which improves Q , and then set $\Xi \rightarrow \Xi'$, and iterate. This is then guaranteed to converge on an optimum, and will often do so in a way which ignores many of the tight local optima that a naive gradient descent algorithm can get caught in.

This is a standard result in mixture models: our approach is complicated by the fact that although it is trivial to find the optimal $\bar{\pi}, \bar{\theta}$, these are not the parameters we are optimising: instead we must optimise the individual expert parameters, which are connected via:

$$\begin{aligned} \bar{\pi}_{p,t} &= \sum_i W_{i,t} \pi_i^p \\ \bar{\mu}_{p,t} &= \sum_i W_{i,t} \mu_i^p \\ \bar{V}_{p,t} &= \sigma_{p,t}^2 = \sum_i W_{i,t} V_i^p \\ \bar{\mathbf{p}}_t &= \sum_i W_{i,t} \mathbf{p}_i \end{aligned} \quad (38)$$

However, the (G)EM approach gives us a useful starting point, because we have wrangled these quantities into *separable linear terms*, which means that they can be independently maximised:

$$\begin{aligned} Q_{\text{weight}}^p(\Xi', \Xi) &= \sum_{t,a} C_{p,t,a}(\Xi) \log(\bar{\pi}'_{p,t}) \\ Q_{\text{shape}}^p(\Xi', \Xi) &= \sum_{t,a} C_{p,t,a}(\Xi) \log(f_p(y_t^a, \bar{\theta}'_{p,t})) \end{aligned} \quad (39)$$

2.3.1 Iterating the Means

For $p < P$, Q_{shape}^p is associated with the mean and variance of the Gaussian modes:

$$Q_S^p = \sum_{t,a} C_{p,t,a} \left[-\frac{1}{2} \log(\bar{V}_{pt}) - \frac{(y_t^a - \bar{\mu}_{pt})^2}{2\bar{V}_{pt}} \right] + \text{const} \quad (40)$$

Then;

$$\begin{aligned} \frac{\partial Q_S^p}{\partial \mu_p^i} &= - \left[\underbrace{\left(\sum_{t,a} \frac{y_t^a W_{i,t} C_{p,t,a}}{\bar{V}_{p,t}} \right)}_{\mathbf{b}_i} - \sum_j \underbrace{\sum_{t,a} \frac{W_{i,t} W_{j,t} C_{p,t,a}}{\bar{V}_{p,t}}}_{M_{ij}} \mu_j^p \right] \\ \frac{\partial Q_S^p}{\partial \mu^p} &= -(\mathbf{b} - M \mu^p) \end{aligned} \quad (41)$$

M is a Gram matrix, and thus positive definite, since $C, V > 0$. The optimal update for the means is therefore:

$$\mu^p \rightarrow M^{-1} \mathbf{b} \quad (42)$$

Iterating the variance. The variance is more complex, and does not have a nice simple form:

$$\frac{\partial Q_S^p}{\partial V_i^p} = \sum_{t,a} C_{p,t,a} W_{i,t} \left[\frac{(y_t^a - \bar{\mu}_{pt})^2}{2\bar{V}_{pt}^2} - \frac{1}{2\bar{V}_{pt}} \right]. \quad (43)$$

This is a concave function, but we recall that we must limit ourselves to $V_i^p > 0$ (and realistically, $V_i^p > \epsilon$, for some finite ϵ). If a positive solution exists, we can reliably converge on it in a handful of Newton steps:

$$V_i^p \rightarrow V_i^p - \frac{1}{\mathcal{C}} \frac{\partial Q_S^p}{\partial V_i^p}, \quad \mathcal{C} = - \sum_{t,a} C_{p,t,a} W_{i,t}^2 \left[\frac{1}{2\bar{V}_{pt}^2} \right], \quad (44)$$

Where $\mathcal{C}$ is not the usual second derivative, but instead the Fisher Scoring derivative, which guarantees each step is in the right direction, at the cost of slightly slower convergence.

Iterating the Weights The values of (μ, V) were coupled across a single mode; we evaluated one mode at a time, of all the experts. The weights are different, as they are coupled across an expert by the requirement that they sum to one. We will therefore update the block $\pi_i = (\pi_i^0, \pi_i^1, \dots, \pi_i^{P-1})$. In order to update the weights, we have to repeat the initial E-M trick, by introducing the contribution that each expert is providing to the overall weight:

$$s_{j,t,p} = \frac{W_{j,t} \pi_j^p}{\bar{\pi}_{p,t}} = \frac{W_{j,t} \pi_j^p}{\sum_i W_{i,t} \pi_i^p} \quad (45)$$

We then average this over all the data points:

$$[\mathbf{c}_j]_p = \sum_{t,a} C_{p,t,a} s_{j,t,p}, \quad (46)$$

Providing a simple update rule:

$$\pi_j \leftarrow \frac{\mathbf{c}_j}{\mathbf{c}_j \cdot \mathbf{1}}. \quad (47)$$

We do note that this *doesn't* provide the optimal set of π_j ; but it is guaranteed to be *better than the previous guess*. This is why we are using the *Generalised-EM* algorithm; we are not perfectly optimising Ξ' , merely improving it, which is a sufficient condition to converge on an optimum.

Iterating the non-parametric parameters The final element is to determine the update rule for $\mathbf{p}_i$, the low resolution non-parametric mode. This has an almost identical form to the update rule for the weights, defining a per-bin contribution, with the only major difference being that r only has a contribution if $y_- \leq y_t^a < y_+$; i.e. it lies within the linear interpolation region that the parameter is associated with.

$$r_{j,t,k}(y) = \frac{W_{j,t} c_k(y) p_j^k}{\sum_{k'} c_{k'}(y) \bar{p}_t^{k'}}, \quad c_k(y) = \begin{cases} 1 - \frac{y - y_{\min} - i(y)\Delta}{\Delta} & k = i(y) \\ \frac{y - y_{\min} - i(y)\Delta}{\Delta} & k = i(y) + 1 \\ 0 & \text{else} \end{cases} \quad (48)$$

Then:

$$[\mathbf{u}_j]_k = \sum_{t,a} C_{P-1,t,a} \times r_{j,t,k}(y_t^a) \quad (49)$$

And finally:

$$\mathbf{p}_i \rightarrow \Delta^{-1} \frac{\mathbf{u}_i}{\mathbf{u}_i \cdot \mathbf{1}} \quad (50)$$

Not implemented!

Note that as of 3/8/26, the non-parametric baseline is not implemented within the code.

2.4 Priors

We mentioned priors earlier, and then omitted them when doing the EM. Priors *can* be included in the EM algorithm, but it is less convenient than in a basic gradient descent.

The Variance, however, is pretty easy (since that does compute the gradient and take optimisation steps). Imposing an inverse gamma prior on $V_{i,p}$ during these steps prevents the variance crashing to zero.

3 The FADE User Manual

This section gives a rough breakdown of the FADE code: how to install, compile and run it.

You will need an internet connection, and some knowledge of how to use a terminal environment (or a well configured [Visual Studio](#) installation). The steps here will work on any POSIX compatible system (Linux or MacOS) – on windows machines you must either [install WSL](#), or have a working `msvc` compiler installed on your machine.

3.1 Installing the Code

The code is hosted on github at <https://github.com/DrFraserGovil/FADE-Emulator/>. It can either be downloaded manually, or cloned in the terminal:

```
>> git clone https://github.com/DrFraserGovil/FADE-Emulator
```

3.2 Compiling the Code

At the present, we do not distribute precompiled binaries, so it is necessary to compile the code on your own machine. You must first [install cmake](#) and ensure that you have a valid compiler (any C++20 compatible compiler is fine).

Then:

```
>> cd FADE-Emulator
>> make
```

You must have an internet connection when you run this command, as it will download⁴ some other software: the [Eigen linear algebra library](#)⁵, and the [Jack Standard Library](#) (JSL), a separate project I have authored that handles much of the ‘application’ code.

It will spit out a bunch of text - as of 03/08/26, these include a number of warnings (in purple) - but it should complete the compilation.

⁴Using CMake FetchContent

⁵If you are familiar with python, this is `numpy.linalg` on steroids

3.2.1 Validating Compilation

Running the command

```
./fade test
```

Places the code into testing mode. If you have a valid python environment (with `matplotlib` installed), it will generate a graphic demonstrating the W_i fractions for a randomly instantiated model.

If you don't have Python - you will get some error messages from your OS, but a `test` directory will have been creating, holding the associated data.

3.3 FADE and JSL

The internal workings of the FADE make heavy use of an existing library I have written, the JSL.

3.3.1 Logging

FADE uses the [custom JSL logger](#) to write the output that you see when writing it. This allows some of the output to be suppressed when you want the code to be 'quiet', or output much more data when you want to know what has gone wrong.

1. `./fade ...` gives the normal output level
2. `./fade --quiet ...` suppresses all output except error messages
3. `./fade --verbose ...` produces lots of output which can be used for diagnostics, but it unsuitable for general use

3.3.2 Configuration

The FADE Emulator has a large number of parameters that can be modified. You may view them by typing:

```
./fade help
```

This activates the help menu, and details which values which can be changed by the user, and a description of what they do. Most settings have a similar name to the names used in this document (though not always).

These settings are grouped into named groups, but this has no meaning for the user, other than helping to group parameters that do similar things together.

Changing the value is done in one of two ways:

Command Line Arguments Running the `./fade` executable and adding extra arguments after it:

```
./fade train -expert 3,3 -model my_file.fde -file training_data.dat
```

This will attempt to launch the training module with:

1. Only 3 experts (the range $3 \rightarrow 3$, see below for multi-model training syntax)
2. Beginning the training at the previous model defined by `my_file.fde`
3. Using the training data stored in `training_data.dat`

All other values will use their default value.

Note that some parameters have multiple aliases which can be used: `-input` and `file` have the same meaning. This is detailed in the `help` menu.

Configuration files As you tweak more and more values, the number of commands given to run the code will get longer and longer. Instead, you can write your data to a config file:

```
// test.cfg
expert 3,3
model my_file.fde
file training_data.dat
(...)

>> ./fade --config test.cfg
```

This loads the data in the config file, and reads it as if it were passed as arguments to the command line.

Mixed-Mode It is possible to use both a config file and command line arguments:

```
>> ./fade --config test.cfg -model other_model.fde
```

In this case, the config file is loaded *first*, and then the command line values are applied, overwriting any values previously set: in the example above, the value passed to `model` is `'other_model.fde'`: the one given at the command line and not `'my_file.fde'` (the one in the config file).

To dash or not two dash?

When specifying command line variables, any number of dashes are allowed: `--flag` and `-flag` are always equivalent. The general advice is to use one dash for short commands (`-f`) and two for long ones (`--model`), but this is not a requirement.

When specifying variables in a config-file, do not use leading dashes.

3.4 Training Models

In order to create a new model, the user must provide **training data**. The expected data format is as follows:

```
x_0 x_1 x_2 (...) x_n z y
// repeat for each y on next lines
```

I.e., a space-delimited enumeration of the position vector $\mathbf{x}_t$, the prior-weighting ζ_t^a and the observed value y_t^a .

If $\mathbf{x}_t$ is one dimensional ($\mathbf{x}_t = x_t$), the data file would look like:

```
//training.data
x_1 z_1 y_1
x_2 z_2 y_2
x_3 z_3 y_3
(...)
```

If you do not know what ζ_t^a is (5), then set these values to 1 (but do not omit them entirely).

Deprecation Imminent

This method of inputting data will be deprecated soon, in favour of one which natively supports $\mathbf{x}$ -clustering. At the moment, clustering is achieved automatically by grouping values of $\mathbf{x}$ which are closer than the `ClusteringRadius` - a user-configured variable.

Once the training data has been generated, the model may be trained:

```
./fade train --file training.data --save my_model.fde --expert 3,6 --dep 2,4
```

This will generate a model called `my_model.fde` which has been trained on the training data. This will simultaneously train the set of submodels for $3 \leq N_e \leq 6$ and $2 \leq N_d \leq 4$ (for a total of 12 individual models).

3.5 Using Models

Once a model has been trained, it can be used to make predictions of the distribution function $p(y|\mathbf{x})$. In order to achieve this, the user must write a file with the following syntax:

```
//qfile.dat
x_0 x_1 x_2 (...) x_n y_1 y_2 y_3 y_4 y_5 (...) y_m
//repeat for each value of X
```

As with the training data, the `x_0 ... x_n` specify the values of $\mathbf{x}$, a point in emulation space to be queried. The values of y form the grid that the model will be evaluated on: so specifying `y_0 y_1 y_2 ..` means “at position $\mathbf{x}$, tell me the probability density at y_0 , then y_1 , then y_2 .” There is no requirement that $\{y_i\}$ be sorted or uniform.

Multiple $\mathbf{x}$ -queries can be entered, with each new $\mathbf{x} - y$ prediction entered on a new line.

The prediction is then made by calling:

```
./fade predict --query qfile.dat --model my_model.fde
```

The output of a prediction is a second data file (with name configured by `query-out`). If the model which was queried was trained on multiple N_e, N_d , then a separate query file (denoted by `[name]_Nd_Ne.dat`) is written for each pair.

For each $\mathbf{x} - y$ specification in the query file, the output file takes the form:

```
New query: x_0 x_1 (...) x_n
y_1 y_2 y_3 (...) y_m
p_1 p_2 p_3 (...) p_m
//repeat for each x-y in query
```

Where:

$$p_i = p^{\text{cns}}(y_i|\mathbf{x}) \quad (51)$$